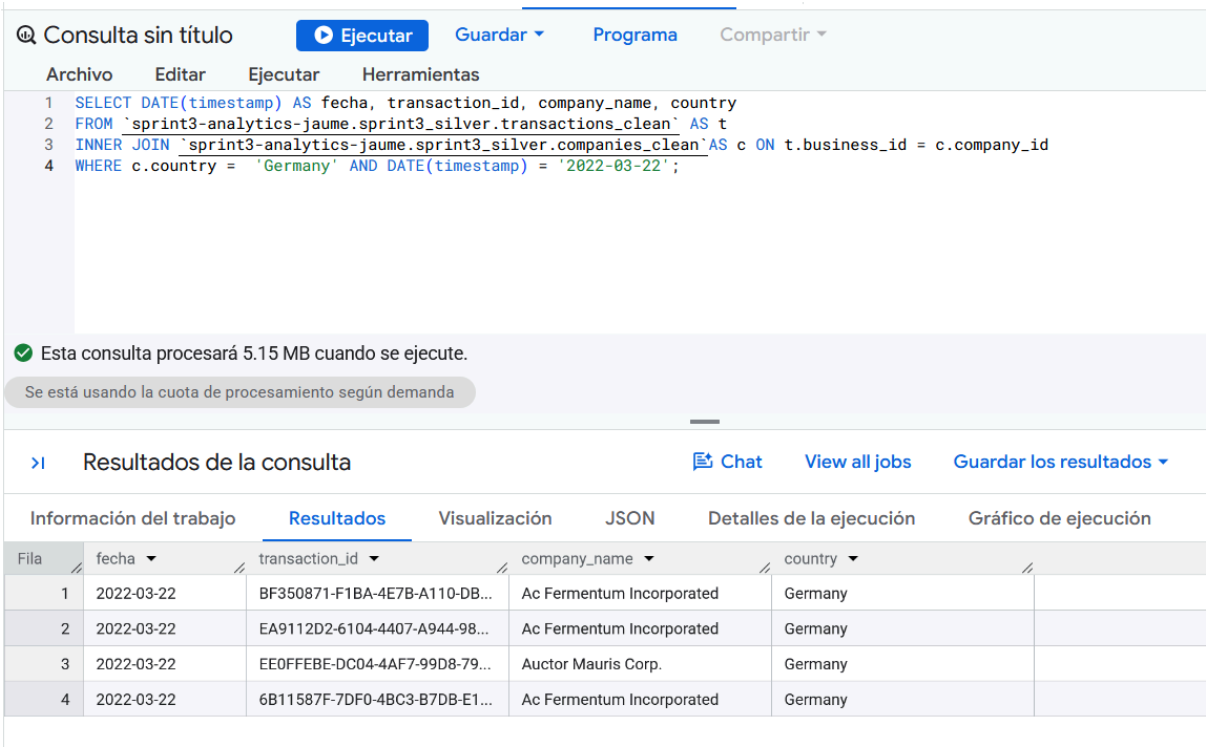
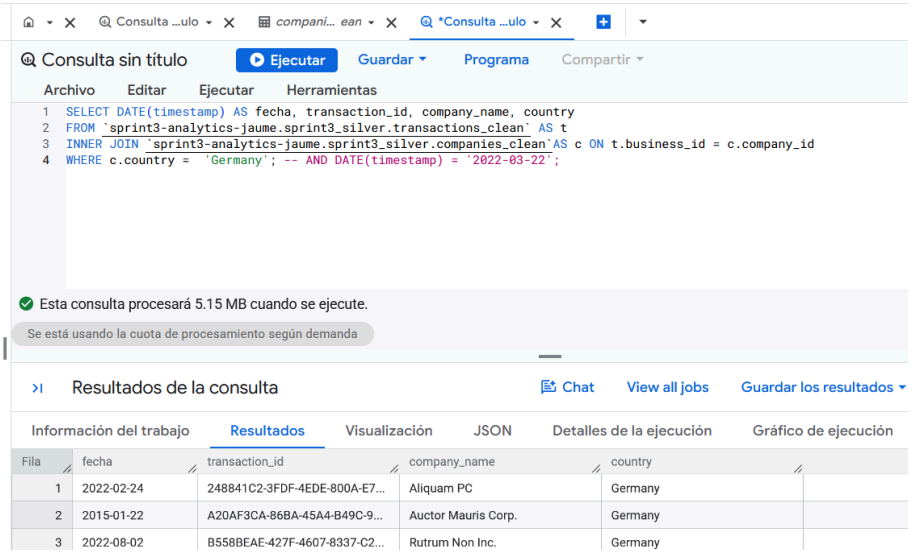
Nivel 1. Ejercicio 1.

Consulta para transacciones de las compañías de Alemania del 22/03/2022:



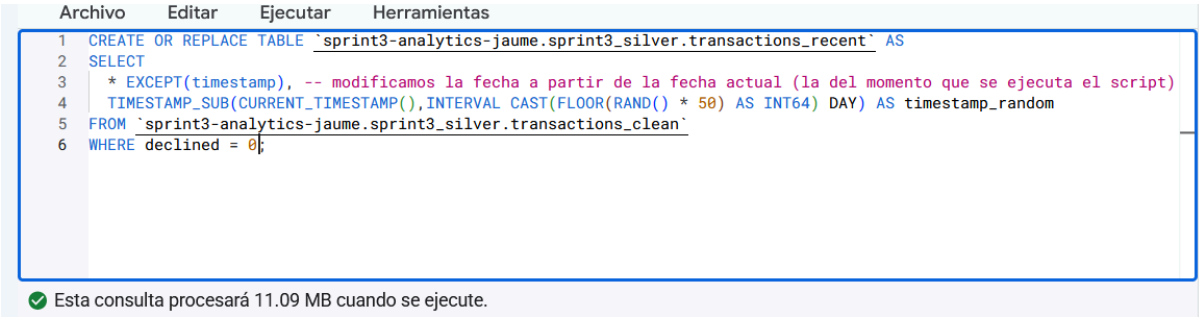
El dry run es de 5.15 MB antes de ejecutar consulta. La estrategia de funcionamiento de BigQuery es leer primero todas las columnas (a diferencia de MySQL que lee por filas). Por tanto, los costes por lectura (bytes) se reducen cuando la tabla se particiona.

Para comprobarlo modificamos la consulta y ampliamos el resultado quitando el filtro de la fecha que nos pide el enunciado y vemos que el Dry Run es exactamente el mismo.

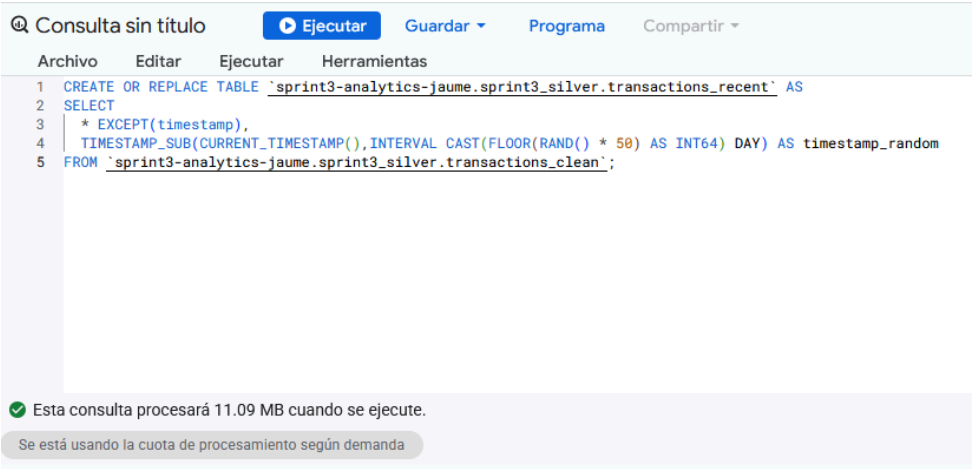


Ejercicio 2.

Creamos la tabla transactions_recent con las transacciones aprobadas (declined = 0) Así ya no tendremos que filtrar de nuevo en el futuro:



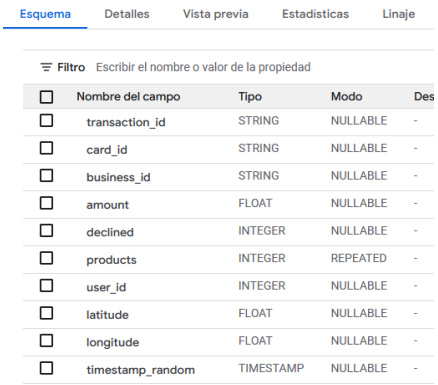
Paso 1:



Generamos un valor aleatorio entre 0 y 50. FLOOR lo redondea y hacemos casting para pasarlo a INTEGER.

TIMESTAMP_SUB hace la resta (substraction) de los días calculados a la fecha del momento de la consulta (CURRENT_TIMESTAMP).

La nueva columna la hemos bautizado como timestamp_random.



Nombre del campo	Tipo	Modo	Des
transaction_id	STRING	NULLABLE	-
card_id	STRING	NULLABLE	-
business_id	STRING	NULLABLE	-
amount	FLOAT	NULLABLE	-
declined	INTEGER	NULLABLE	-
products	INTEGER	REPEATED	-
user_id	INTEGER	NULLABLE	-
latitude	FLOAT	NULLABLE	-
longitude	FLOAT	NULLABLE	-
timestamp_random	TIMESTAMP	NULLABLE	-

Creamos la tabla optimizada en gold:

Archivo Editar Ejecutar Herramientas

```sql
CREATE OR REPLACE TABLE `sprint3-analytics-jaume.sprint3_gold.fact_transactions_optimized`
PARTITION BY DATE(timestamp_random) --particionamos a partir de la fecha
CLUSTER BY business_id AS --ordenamos cada particion a partir de la id de la compañia
SELECT *
FROM `sprint3-analytics-jaume.sprint3_silver.transactions_recent`
WHERE declined = 0;
```

Esta consulta procesará 11.82 MB cuando se ejecute.

Bigquery nos confirma que la tabla está particionada:

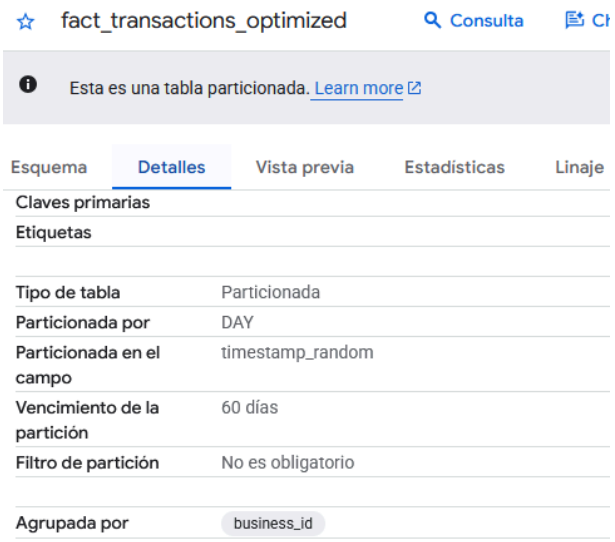
fact_transactions_optimized Consulta Chat Abrir en Compartir Copiar Instantánea Borrar Exportar o sincronizar

Esta es una tabla particionada. Learn more

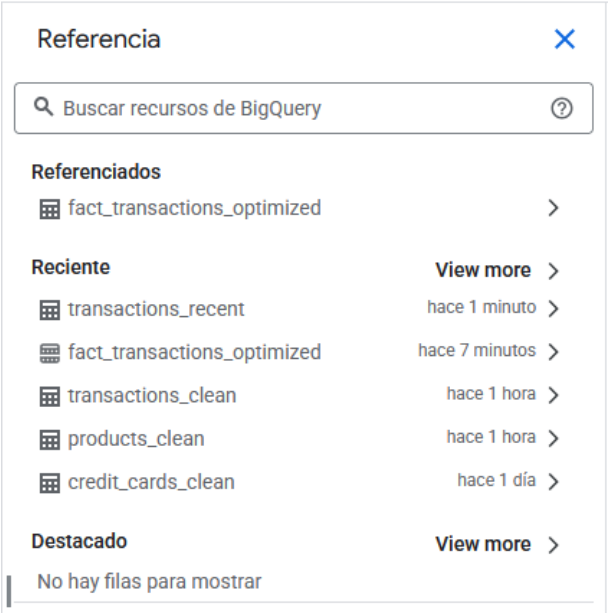
Esquema Detalles Vista previa Estadísticas Linaje Perfil de datos Calidad de los datos

Fila	transaction_id	card_id	business_id	amount	declined	products	user_id	latitude	longitude	timestamp_random
1	D8C78EB3-B149-4E02-9A7D-28...	CcS-4930	b-2222	653.67	0	92	349	60.0596927...	18.9289490...	2026-09-02 11:25:39.098113 UTC
						98				
						12				
						53				
						28				
2	66D1DC53-6C53-46DF-9649-81...	CcS-7313	b-2222	458.66	0	25	2732	51.5710837...	10.2159067...	2026-09-02 11:25:39.098113 UTC
						33				
						17				
						47				
3	46FD95EE-E58A-48FE-8CB1-F6...	CcS-4892	b-2222	379.81	0	37	311	41.5279140...	12.5406924...	2026-09-02 11:25:39.098113 UTC
						64				
						25				
4	BC99B4D5-6631-403B-BA3B-C5...	CcS-6449	b-2222	318.44	0	13	1868	41.5095432...	12.3605916...	2026-09-02 11:25:39.098113 UTC
						78				
5	45727E3B-07C5-4415-83BB-50...	CcS-4919	b-2222	136.96	0	76	338	60.4777126...	18.9181370...	2026-09-02 11:25:39.098113 UTC
						4				
6	363A6478-EBC9-49CF-9ED3-77...	CcS-5035	b-2222	325.49	0	29	454	52.2915945...	19.1671805...	2026-09-02 11:25:39.098113 UTC
						74				
7	AD17A138-1EF8-4732-856F-23...	CcS-8736	b-2222	289.31	0	37	4155	60.6163598...	18.5694858...	2026-09-02 11:25:39.098113 UTC
						17				

Más información en la pestaña de detalles en el apartado tipo de tabla:

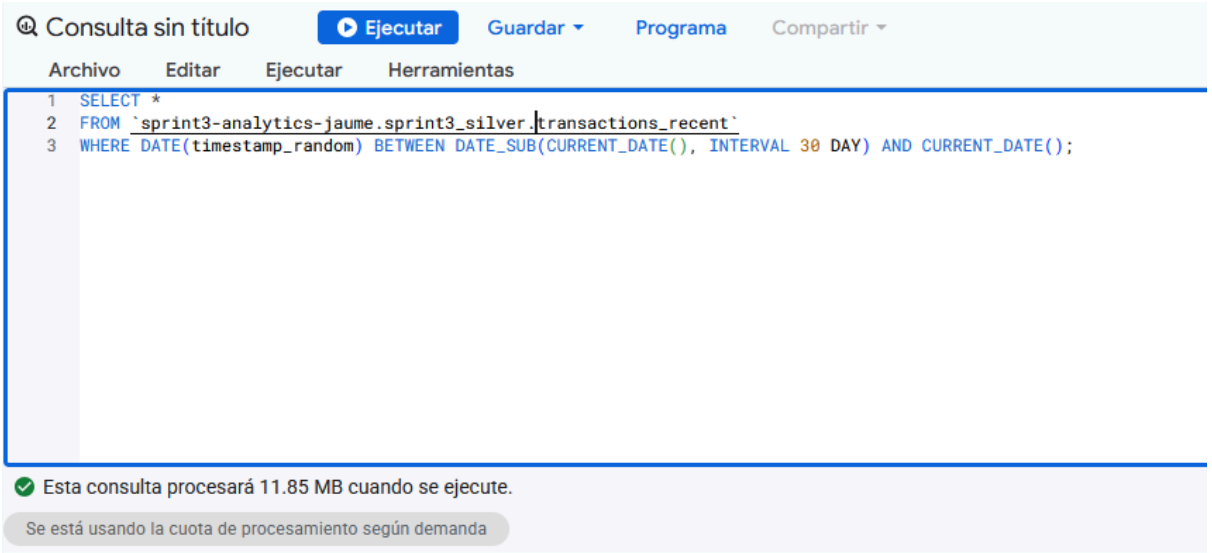


También se puede ver en el apartado de referencia de las tablas (panel de la derecha en big query). La tabla particionada tiene un icono específico:

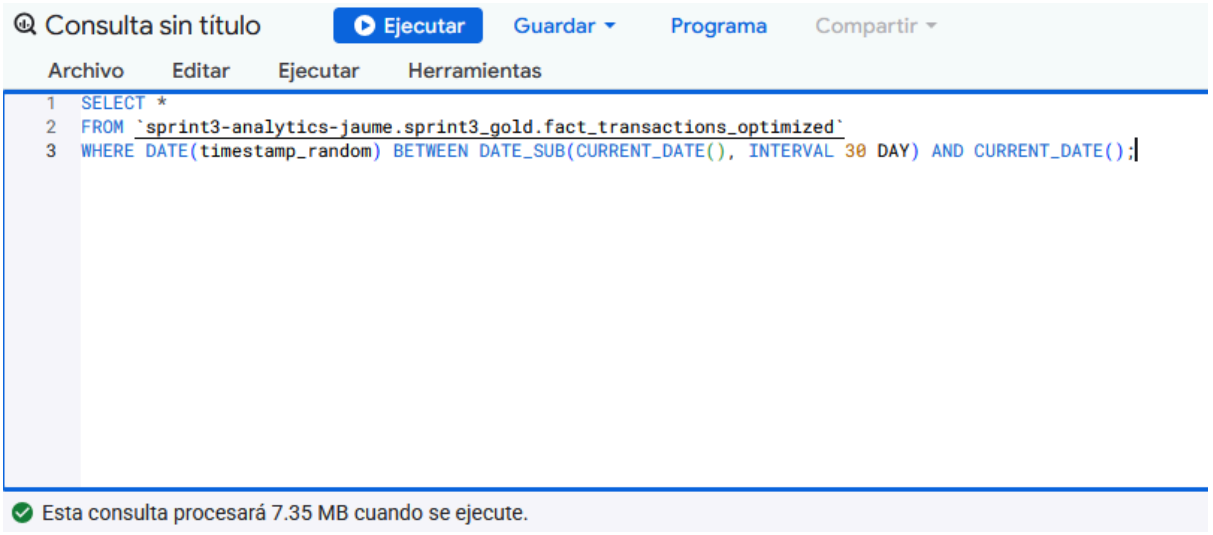


Ejercicio 3.

Realizamos consulta a transactions sin optimizar (tabla transactions_recent):



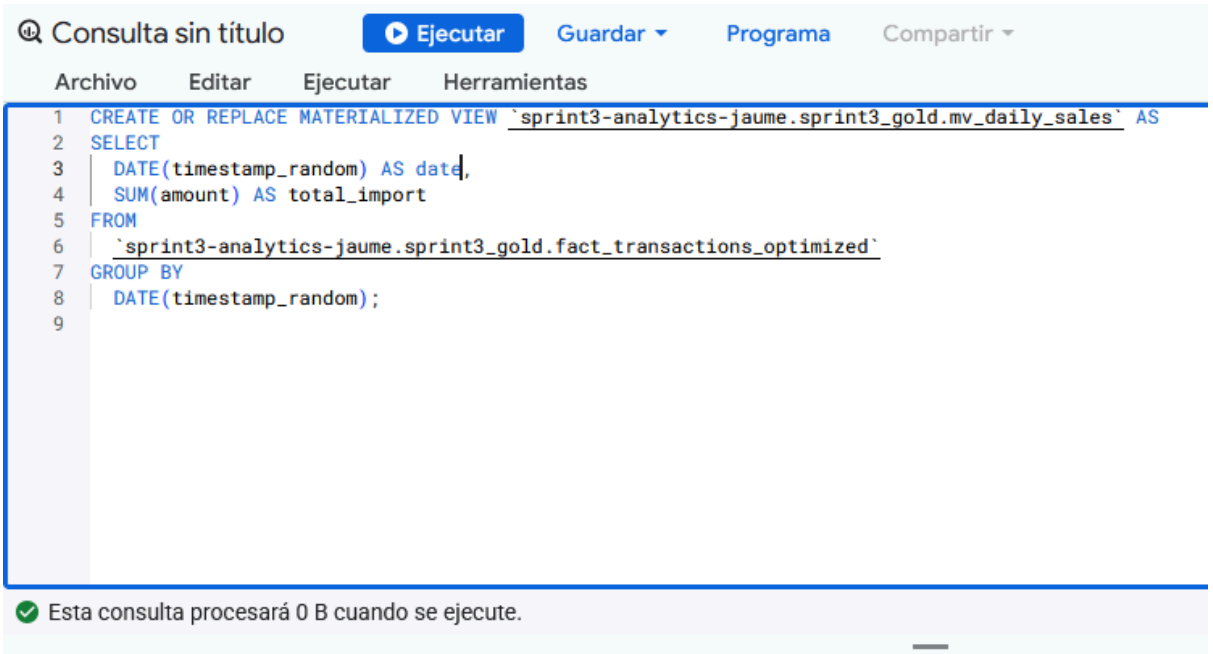
Y la misma consulta sobre transactions optimizada (tabla fact_transactions_optimized):



Podemos ver que disminuimos el tamaño de la consulta en casi un 38% si miramos sobre la tabla optimizada.
Dado que trabajamos con la tabla *fact_transactions_optimized*, sabemos que tenemos todas las transacciones realizadas (las *declined* se quedaron fuera cuando creamos la tabla).

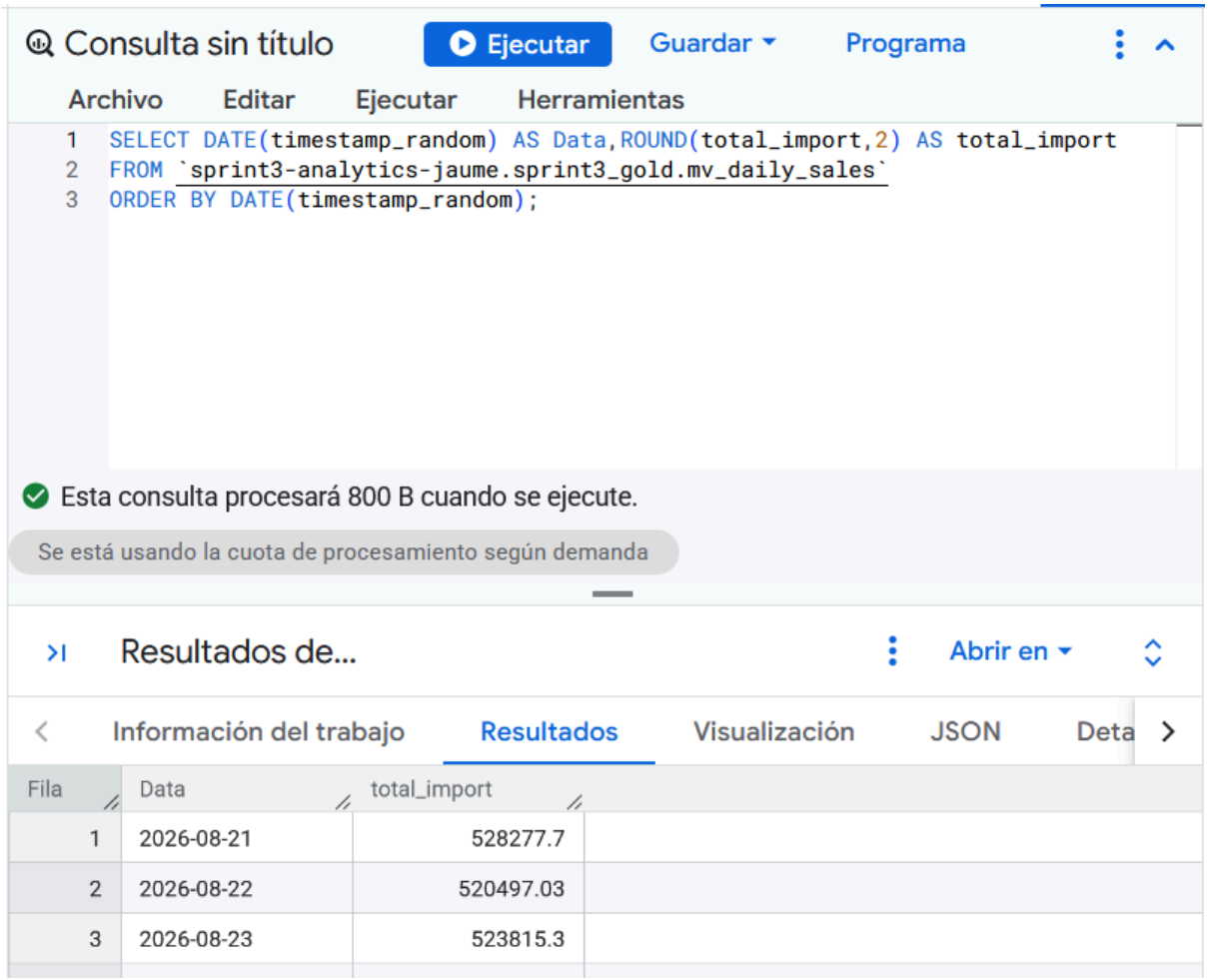
Ejercicio 4.

Creamos la MATERIALIZED VIEW:



Las *materialized views* tienen por defecto la actualización de la vista cada vez que se actualiza la tabla (enable_refresh = true). Por lo que cada vez que se consulte la vista, siempre se tendrá una visión al día de los datos.

Y los datos de la vista se obtienen a partir de la siguiente consulta (que tiene un coste despreciable):

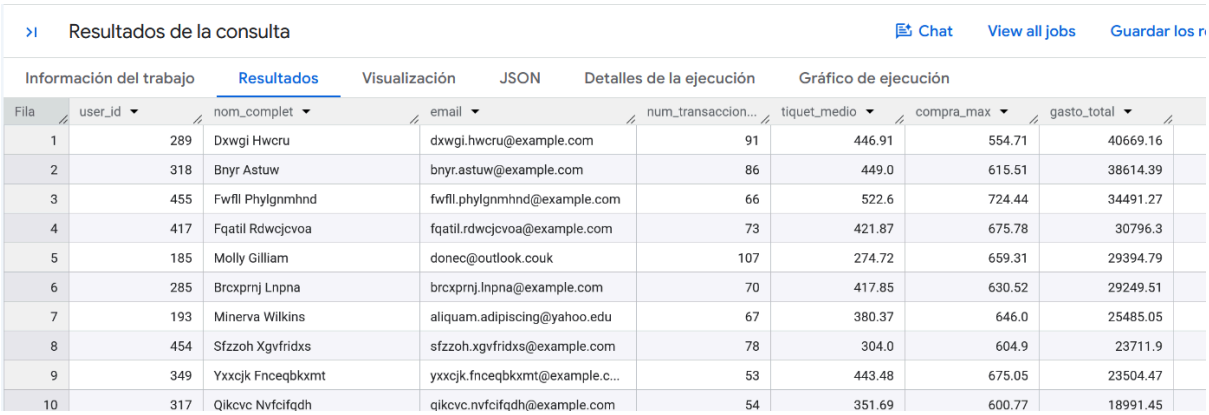


Nivel 2. Ejercicio 1.

Creamos la CTE con el nombre que nos da el enunciado y hacemos el inner join con la tabla de usuarios:



Y en el output obtenemos la tabla según nos pide el enunciado con user_id, nom_complet, email, num_compres, tiquet_mig, max_compra, total_gastat y orden descendente en gasto total:



Fila	user_id	nom_complet	email	num_transaccion...	tiquet_medio	compra_max	gasto_total
1	289	Dxwgi Hwcru	dxwgi.hwcru@example.com	91	446.91	554.71	40669.16
2	318	Bnyr Astuw	bnyr.astuw@example.com	86	449.0	615.51	38614.39
3	455	Fwfll Phylgnmhnd	fwfll.phylgnmhnd@example.com	66	522.6	724.44	34491.27
4	417	Fqatil Rdwcjcvoa	fqatil.rdwcjcvoa@example.com	73	421.87	675.78	30796.3
5	185	Molly Gilliam	donec@outlook.couk	107	274.72	659.31	29394.79
6	285	Brcxprnj Lnpna	brcxprnj.lnpna@example.com	70	417.85	630.52	29249.51
7	193	Minerva Wilkins	aliquam.adipiscing@yahoo.edu	67	380.37	646.0	25485.05
8	454	Sfzzoh Xgvfridxs	sfzzoh.xgvfridxs@example.com	78	304.0	604.9	23711.9
9	349	Yxxcjk Fnceqbkxmt	yxxcjk.fnceqbkxmt@example.c...	53	443.48	675.05	23504.47
10	317	Qikcvc Nvfcifqdh	qikcvc.nvfcifqdh@example.com	54	351.69	600.77	18991.45

Ejercicio 2.

Creamos una CTE para obtener la columna con el valor del día anterior al de consulta

Consulta sin título

Ejecutar | Guardar | Programa | Compartir

Archivo | Editar | Ejecutar | Herramientas

```sql
SELECT
  DATE(timestamp_random) AS fecha,
  ROUND(SUM(total_import),2)
FROM `sprint3-analytics-jaume.sprint3_gold.mv_daily_sales`
GROUP BY fecha
ORDER BY fecha ASC;
```

Esta consulta procesará 800 B cuando se ejecute.

A continuación transformamos esta consulta en una CTE a la que llamaremos ventas. A mí me ayuda a aclarar el código. Luego implementamos la función ventana para poder comparar cada día con el día anterior sobre esta CTE.

Archivo | Editar | Ejecutar | Herramientas

```sql
WITH ventas AS (
  SELECT
    timestamp_random,
    total_import AS vendes_avui,
    LAG(total_import) OVER (ORDER BY timestamp_random)    AS vendes_ahir
  FROM `sprint3-analytics-jaume.sprint3_gold.mv_daily_sales`
)
SELECT
  DATE(timestamp_random) AS data,
  ROUND(vendes_avui, 2) AS vendes_avui,
  ROUND(vendes_ahir, 2) AS vendes_ahir,
  ROUND(SAFE_DIVIDE(vendes_avui - vendes_ahir, vendes_ahir) * 100, 2) AS diff_percentual -- usamos SAFE_DIVIDE para evitar divisiones por 0
FROM ventas
ORDER BY data;
```

Esta consulta procesará 800 B cuando se ejecute.

Se está usando la cuota de procesamiento según demanda

Resultados de la consulta | Chat | View all jobs | Guardar los resultados | Abrir en

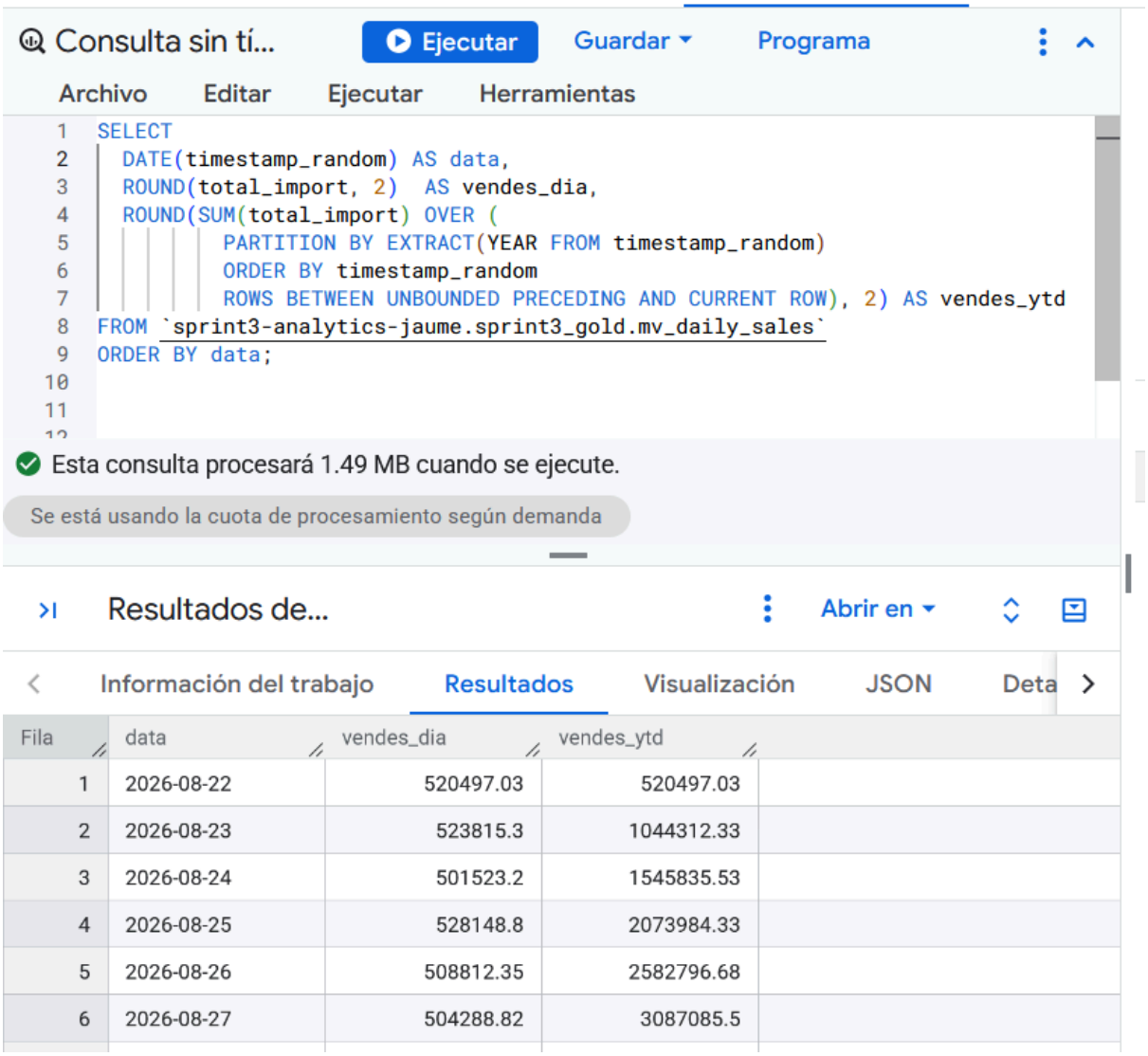
Información del trabajo | Resultados | Visualización | JSON | Detalles de la ejecución | Gráfico de ejecución

Fila	data	vendes_avui	vendes_ahir	diff_percentual
1	2026-08-21	528277.7	*null*	*null*
2	2026-08-22	520497.03	528277.7	-1.47
3	2026-08-23	523815.3	520497.03	0.64
4	2026-08-24	501523.2	523815.3	-4.26

El porcentaje es la decisión de las ventas del dia menos las del dia anterior dividido por estas mismas ventas. Para evitar que la función se pare por error en caso de encontrar algún cero o null usaremos el SAFE_DIVIDE. Así aseguramos la compleción de la ejecución de la función.
En el output obtenido tenemos dos nuevas columnas vendes_ahir, que corresponde a la primera función ventana y diferencia_diaria que corresponde a la diferencia en percentil del día de esa fila con el anterior, redondeado a 2 decimales.

Ejercicio 3

Creamos de nuevo una CTE para tener la suma diaria sobre la materialized view. A continuación implementamos función ventana para poder hacer el sumatorio acumulado de las ventas al mismo año (a partir del BY EXTRACT(YEAR FROM fecha).



Fila	data	vendes_dia	vendes_ytd
1	2026-08-22	520497.03	520497.03
2	2026-08-23	523815.3	1044312.33
3	2026-08-24	501523.2	1545835.53
4	2026-08-25	528148.8	2073984.33
5	2026-08-26	508812.35	2582796.68
6	2026-08-27	504288.82	3087085.5

En el output obtenemos las 2 columnas que nos pide el enunciado, aparte de la fecha, con las ventas del dia y el acumulado que se va incrementando día a día

Ejercicio 4

La estrategia que nos propone el enunciado es usar QUALIFY para discriminar las transacciones más allá de la tercera que haya podido hacer cada usuario.

Debemos pues crear una consulta en forma de CTE que nos haga una primera criba para quedarnos con las 3 primeras transacciones.
A continuación hacemos un INNER JOIN de la CTE con la tabla de usuarios para poder tener el nombre completo.

Archivo Editar Ejecutar Herramientas

```sql
WITH tres_prim_compras AS (
  SELECT
    user_id,
    timestamp_random AS data_compra,
    amount AS import_compra,
    -- Numeramos únicamente las compras no rechazadas
    ROW_NUMBER() OVER (
      PARTITION BY user_id
      ORDER BY timestamp_random ASC
    ) AS num_compra
  FROM `sprint3-analytics-jaume.sprint3_silver.transactions_recent`
  WHERE declined = 0 -- Garantiza que las compras no son rechazadas
  QUALIFY num_compra <= 3
)

SELECT
  us.user_id,
  CONCAT(us.name, ' ', us.surname) AS nom_complet,
  us.email,
  MAX(CASE WHEN t.num_compra = 3 THEN t.data_compra END) AS data_3a_compra,
  MAX(CASE WHEN t.num_compra = 3 THEN t.import_compra END) AS import_3a_compra,
  ROUND(AVG(t.import_compra), 2) AS mitjana_3_primeres

FROM tres_prim_compras AS t
INNER JOIN `sprint3-analytics-jaume.sprint3_silver.users_combined` AS us
  ON t.user_id = us.user_id
GROUP BY
  us.user_id, us.name, us.surname, us.email
HAVING COUNT(t.num_compra) = 3
ORDER BY data_3a_compra ASC;
```

Esta consulta procesará 3.3 MB cuando se ejecute.

Buscamos la fecha y el importe de la tercera compra a partir del índice num_compra y la media a partir de la suma de los importes de la CTE, que ya están acotados como hemos dicho por el QUALIFY. Controlamos que las transacciones no hayan sido rechazadas también.
En la función CASE escribimos la fecha o el importe de la columna correspondiente en el caso de que sea 3 su número de orden. Hemos añadido MAX al inicio de la función para quedarnos solo con los valores del orden 3, y para los de orden 1 y 2 descartará los null. Esto es importante por que al final hacemos una agrupación por usuarios y habrian muchas filas en las que la condición será falsa.

Nivel 3. Ejercicio 1.

Para aplanar la tabla con las ids en forma de array vamos a seguir las instrucciones del enunciado. Primero crearemos la tabla en el dataset GOLD. A continuación creamos una CTE para hacer este paso donde usaremos la función UNNEST de las ids de los productos en la tabla transactions. Con ello desapilamos las array para conseguir la “flat table”. Lo haremos sobre la columna products: CROSS JOIN UNNEST(products). Lo que hacemos es escribir una fila para cada id de la array copiando los valores de las otras columnas.

A continuación hacemos un *inner join* con la tabla de products para aunar las ids con el nombre del producto.

Consulta sin título | Ejecutar | Guardar | Programa | Compartir

Archivo Editar Ejecutar Herramientas

```sql
WITH tres_prim_compras AS (
  SELECT
    user_id,
    timestamp_random AS data_compra,
    amount AS import_compra,
    -- Numeramos las compras de las transacciones
    ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY timestamp_random ASC) AS num_compra
  FROM `sprint3-analytics-jaume.sprint3_silver.transactions_recent`
-- QUALIFY sobre la función ventana para discrimanr todas las compras posteriores a la 3a compra
-- ahorramos el cálculo innecesario ya que nos piden trabajar SOLO sobre las 3 primeras compras
  QUALIFY num_compra <= 3)

SELECT
  us.user_id,
  CONCAT(us.name,' ',us.surname) AS nom_complet,
  us.email,
  -- Data e importe de la 3a compra exacta
MAX(CASE WHEN t.num_compra = 3 THEN t.data_compra END) AS data_3a_compra,
MAX(CASE WHEN t.num_compra = 3 THEN t.import_compra END) AS import_3a_compra,
  -- AVG de las 3 primeras compras
  ROUND(AVG(t.import_compra), 2) AS mitjana_3_primeres

FROM tres_prim_compras AS t
INNER JOIN `sprint3-analytics-jaume.sprint3_silver.users_combined` AS us ON t.user_id = us.user_id
GROUP BY us.user_id,us.name,us.surname,us.email
-- Con el HAVING COUNT nos quedamos con los usuarios que tienen las 3 compras hechas
HAVING COUNT(t.num_compra) = 3
ORDER BY data_3a_compra ASC;
```

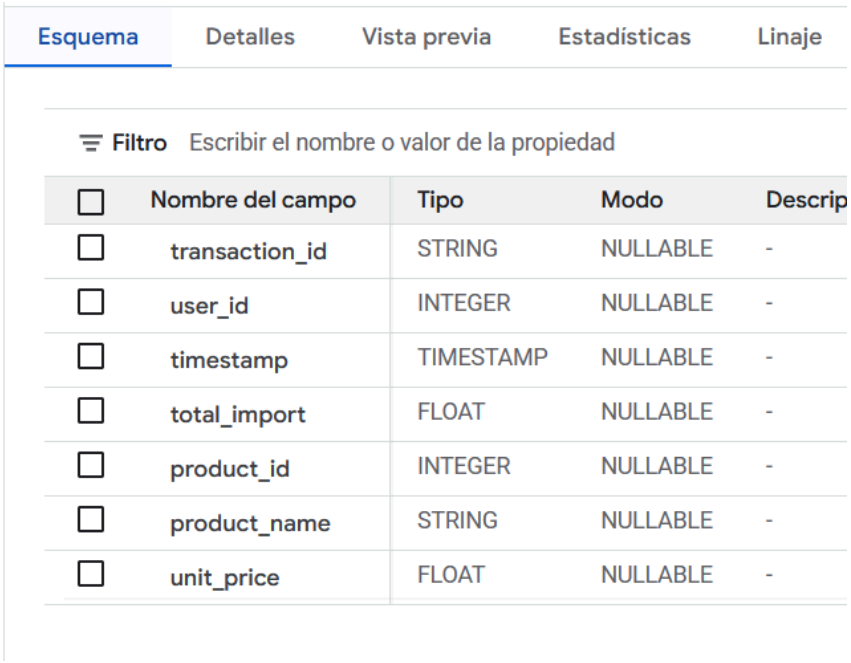
Esta consulta procesará 2.54 MB cuando se ejecute.

La tabla queda sin el array de los productos y con el nombre al lado de cada id:

Fila	transaction_id	user_id	timestamp	total_import	product_id	product_name	unit_price
1	F16375F4-84AC-4A81-B8BC-C5...	97	2022-12-13 05:28:08 UTC	484.69	14	Direwolf	147.53
2	6F7EE539-241D-4FCC-86FD-93...	151	2019-10-25 09:57:35 UTC	314.68	14	Direwolf	147.53
3	F97CE82D-EC75-4A56-BAB2-D...	171	2016-05-28 12:40:46 UTC	371.76	14	Direwolf	147.53
4	D1063158-59F1-4B21-AF26-A9...	297	2024-05-10 21:30:47 UTC	322.33	14	Direwolf	147.53
5	1790ABDF-A05C-4FD1-AFFD-2...	397	2018-07-02 02:21:58 UTC	237.43	14	Direwolf	147.53
6	B13F4273-3148-4392-AE15-9C...	521	2016-06-27 14:12:31 UTC	341.12	14	Direwolf	147.53
7	8B318C3E-ABF1-4AF3-9ECE-53...	833	2023-12-31 08:34:22 UTC	320.46	14	Direwolf	147.53
8	A1D96566-9849-4511-A840-26...	846	2017-02-10 22:48:50 UTC	237.15	14	Direwolf	147.53
9	05B48864-7E82-475D-9951-8B...	1095	2018-02-15 14:16:16 UTC	269.44	14	Direwolf	147.53
10	61518525-3748-44BC-85C5-5C...	1132	2021-06-26 02:38:09 UTC	187.26	14	Direwolf	147.53
11	42B9BEA2-3314-48C7-9CEB-6C...	1192	2024-02-11 20:12:59 UTC	329.67	14	Direwolf	147.53
12	3CCA0D1D-8B92-44DD-8097-C...	1293	2016-01-17 20:36:56 UTC	200.84	14	Direwolf	147.53
13	5909E565-1864-434A-B2E3-03...	1364	2016-08-04 22:29:44 UTC	357.0	14	Direwolf	147.53
14	D1D18E78-A811-45D8-8DD9-0A...	1368	2016-09-19 10:04:25 UTC	257.83	14	Direwolf	147.53
15	8DB8FA93-9B86-4C83-B75B-59...	1421	2024-02-10 08:06:22 UTC	466.15	14	Direwolf	147.53
16	D85C144A-DD87-4B4C-B2C8-1...	1550	2015-12-07 10:20:16 UTC	221.66	14	Direwolf	147.53
17	795AD321-EF6F-4721-A70C-8C...	1659	2023-07-10 01:50:31 UTC	147.53	14	Direwolf	147.53

Resultados por página: 50 1 – 50 de 253391

El esquema de la tabla es el que sigue:

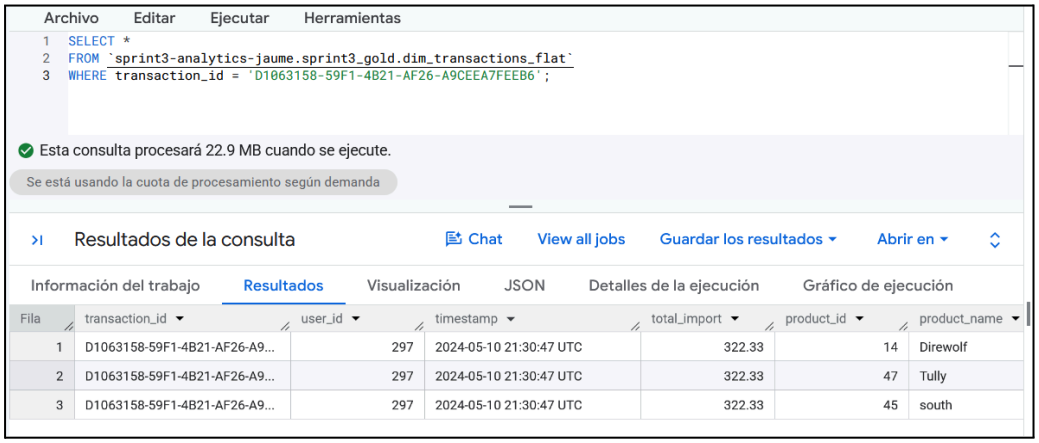


Esquema | Detalles | Vista previa | Estadísticas | Linaje

Filtro Escribir el nombre o valor de la propiedad

Nombre del campo	Tipo	Modo	Descrip
transaction_id	STRING	NULLABLE	-
user_id	INTEGER	NULLABLE	-
timestamp	TIMESTAMP	NULLABLE	-
total_import	FLOAT	NULLABLE	-
product_id	INTEGER	NULLABLE	-
product_name	STRING	NULLABLE	-
unit_price	FLOAT	NULLABLE	-

Si probamos con una *transaction_id* cualquiera podemos ver que está desglosado cada producto de la transacción con su nombre y el total_import del pedido detallado.



Archivo Editar Ejecutar Herramientas

```
SELECT *
FROM `sprint3-analytics-jaume.sprint3_gold.dim_transactions_flat`
WHERE transaction_id = 'D1063158-59F1-4B21-AF26-A9CEEA7FEEB6';
```

Esta consulta procesará 22.9 MB cuando se ejecute.

Se está usando la cuota de procesamiento según demanda

Resultados de la consulta | Chat | View all jobs | Guardar los resultados | Abrir en

Información del trabajo | Resultados | Visualización | JSON | Detalles de la ejecución | Gráfico de ejecución

Fila	transaction_id	user_id	timestamp	total_import	product_id	product_name
1	D1063158-59F1-4B21-AF26-A9...	297	2024-05-10 21:30:47 UTC	322.33	14	Direwolf
2	D1063158-59F1-4B21-AF26-A9...	297	2024-05-10 21:30:47 UTC	322.33	47	Tully
3	D1063158-59F1-4B21-AF26-A9...	297	2024-05-10 21:30:47 UTC	322.33	45	south

Ejercicio 2.

Al tener la tabla con las ids desapilados (sin array) ahora podemos hacer consultas simples con toda la granularidad de la información al completo.

Hacemos un count de las filas agrupadas por ids y nombre y limitamos el número a 5 y ordenamos de forma descendente:

Archivo Editar Ejecutar Herramientas

```sql
SELECT product_id, product_name, COUNT(*) AS cantidad_unds
FROM `sprint3-analytics-jaume.sprint3_gold.dim_transactions_flat`
GROUP BY product_id, product_name
ORDER BY cantidad_unds DESC
LIMIT 5;
```

Esta consulta procesará 5.99 MB cuando se ejecute.

Se está usando la cuota de procesamiento según demanda

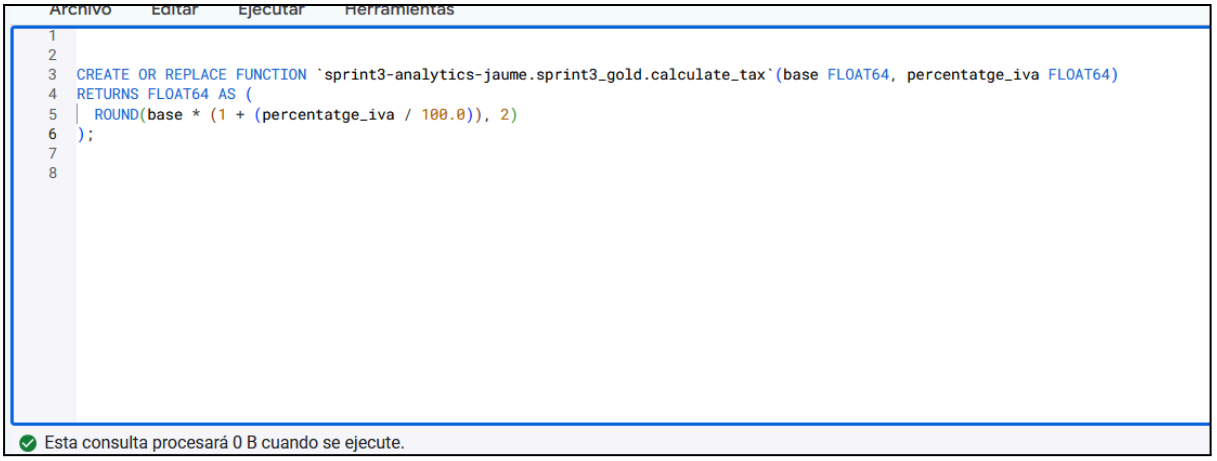
Resultados de la consulta Chat View all jobs Guardar los resultad

Información del trabajo | Resultados | Visualización | JSON | Detalles de la ejecución

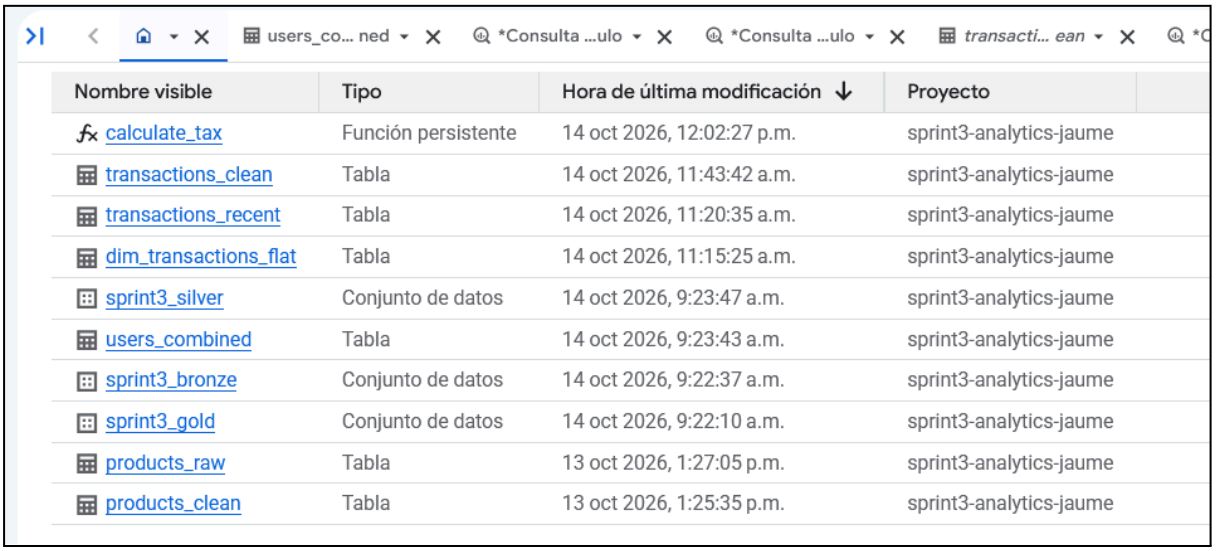
Fila	product_id	product_name	cantidad_unds
1	52	riverlands the duel	2654
2	29	Tully maester Tarly	2635
3	21	duel Direwolf	2609
4	16	the duel warden	2608
5	66	mustafar jinn	2601

Ejercicio 3

Primero creamos la UDF. No introducimos el valor del IVA, lo dejamos como un parámetro que se leerá cuando se llame a la función:

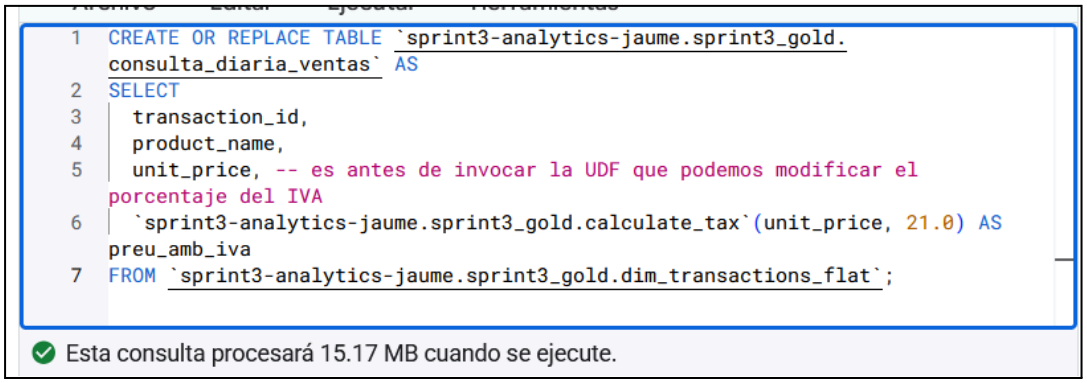


La función creada queda listada en el proyecto:

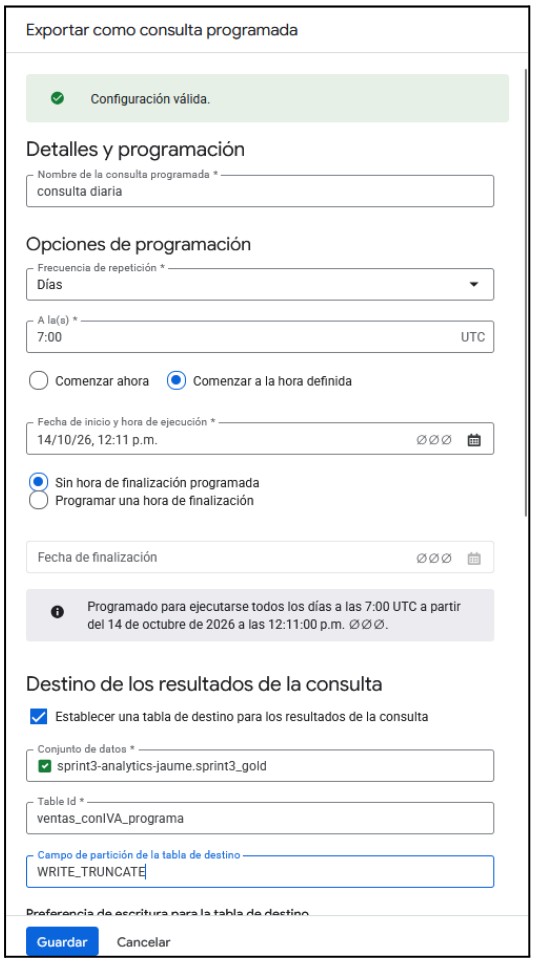


Nombre visible	Tipo	Hora de última modificación	Proyecto
calculate_tax	Función persistente	14 oct 2026, 12:02:27 p.m.	sprint3-analytics-jaume
transactions_clean	Tabla	14 oct 2026, 11:43:42 a.m.	sprint3-analytics-jaume
transactions_recent	Tabla	14 oct 2026, 11:20:35 a.m.	sprint3-analytics-jaume
dim_transactions_flat	Tabla	14 oct 2026, 11:15:25 a.m.	sprint3-analytics-jaume
sprint3_silver	Conjunto de datos	14 oct 2026, 9:23:47 a.m.	sprint3-analytics-jaume
users_combined	Tabla	14 oct 2026, 9:23:43 a.m.	sprint3-analytics-jaume
sprint3_bronze	Conjunto de datos	14 oct 2026, 9:22:37 a.m.	sprint3-analytics-jaume
sprint3_gold	Conjunto de datos	14 oct 2026, 9:22:10 a.m.	sprint3-analytics-jaume
products_raw	Tabla	13 oct 2026, 1:27:05 p.m.	sprint3-analytics-jaume
products_clean	Tabla	13 oct 2026, 1:25:35 p.m.	sprint3-analytics-jaume

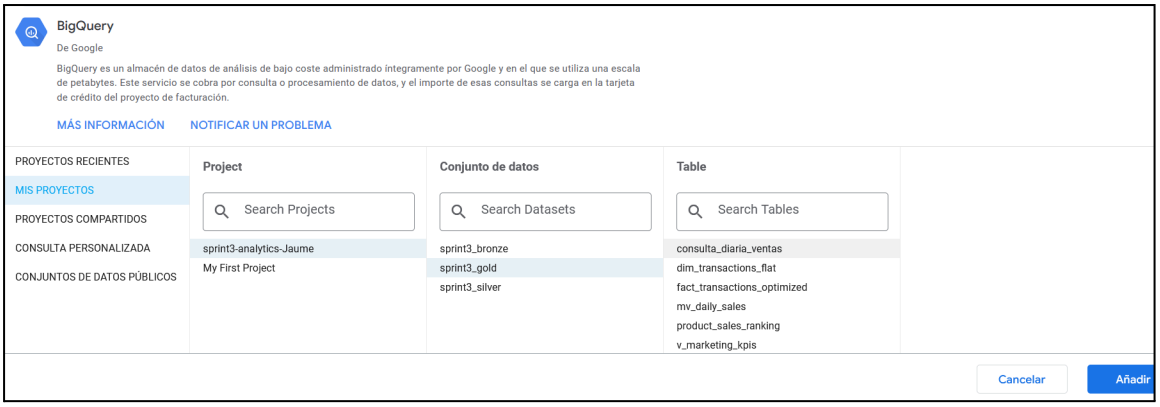
Lanzamos la consulta con la función creada:



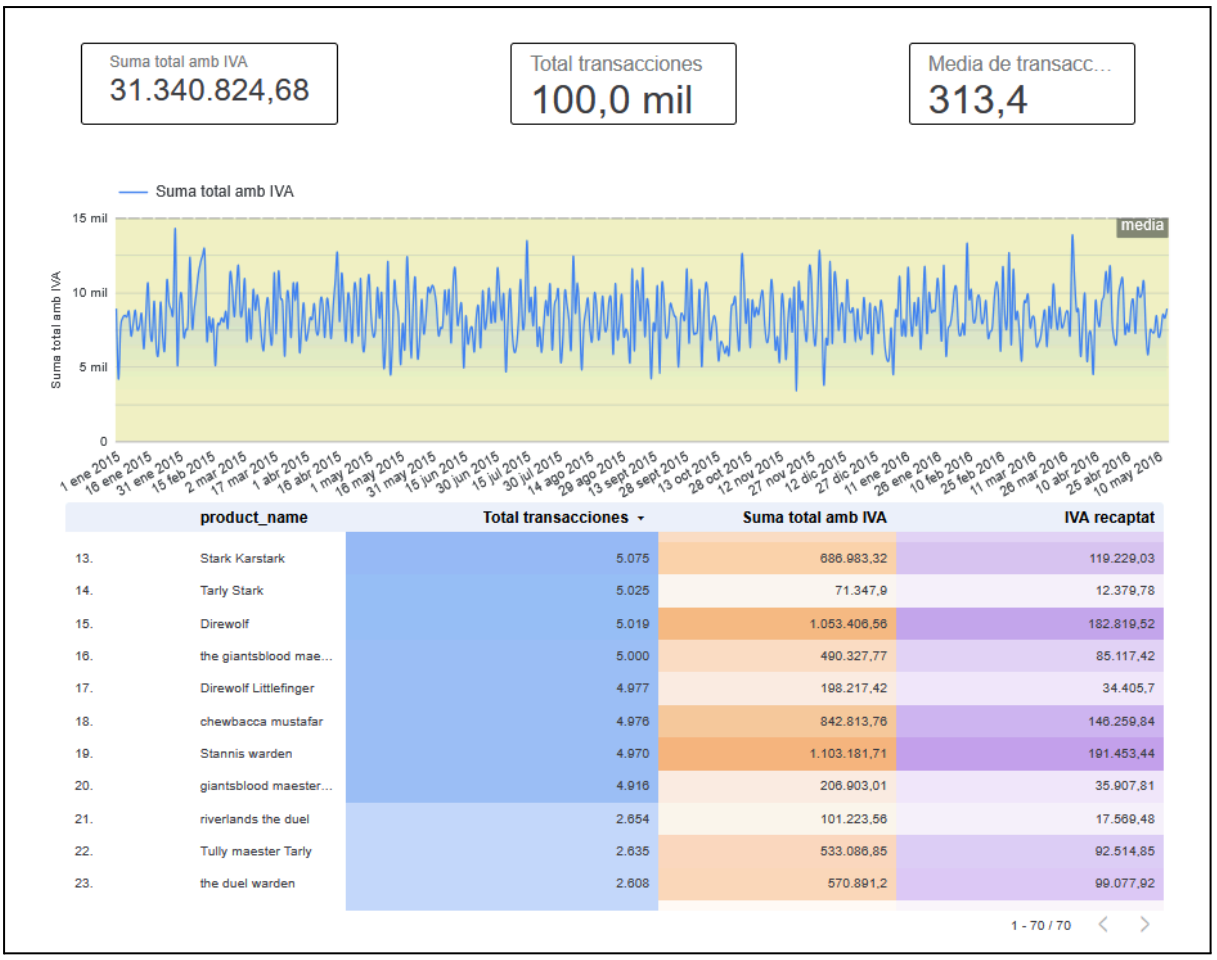
Y hacemos la programación de la consulta para cada dia a las 7:00 de la mañana a partir de la fecha actual:



Conectamos BigQuery con Data Studio a partir de nuestro proyecto hasta la tabla creada en la consulta programada:



Y generamos el dashboard con los detalles según el enunciado:



	product_name	Total transacciones	Suma total amb IVA	IVA recaptat
13.	Stark Karstark	5.075	686.983,32	119.229,03
14.	Tarly Stark	5.025	71.347,9	12.379,78
15.	Direwolf	5.019	1.053.406,56	182.819,52
16.	the giantsblood mae...	5.000	490.327,77	85.117,42
17.	Direwolf Littlefinger	4.977	198.217,42	34.405,7
18.	chewbacca mustafar	4.976	842.813,76	146.259,84
19.	Stannis warden	4.970	1.103.181,71	191.453,44
20.	giantsblood maester...	4.916	206.903,01	35.907,81
21.	riverlands the duel	2.654	101.223,56	17.569,48
22.	Tully maester Tarly	2.635	533.086,85	92.514,85
23.	the duel warden	2.608	570.891,2	99.077,92

1 - 70 / 70

No he conseguido tener la línea de tendencia tal como propone el enunciado. Lo he probado TODO. Pero en los menús de estilo del gráfico no me aparece la posibilidad de añadirla.